

Praktikum Basis Data

Modul 09

Scripting Procedure and Function



Oleh:

Khosy Albuchary

103122400030

**PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026**

JURNAL

1. Replikasi Skema (CREATE TABLE)

```
CREATE TABLE employees AS  
SELECT * FROM hremployees;  
  
SELECT * FROM employees;
```

EMPLOYEE_ID	LAST_NAME	EMAIL	SALARY	COMMISSION_PCT	HIRE_DATE	DEPARTMENT_ID
1	101 Kochhar	NHOCHHAR	17000	(null)	21-SEP-21	90
2	102 De Haan	LDEHAAN	17000	(null)	13-JAN-22	90
3	103 Hunold	AHUNOLD	9000	(null)	03-JAN-22	60
4	104 Ernst	AERNST	6000	(null)	21-MAY-23	60
5	107 Lorentz	DLORENTZ	4200	(null)	07-FEB-24	60
6	145 Russell	JRUSSELL	14000	0.4	01-OCT-21	80
7	146 Partners	KPARTNER	13500	0.3	05-JAN-21	80
8	147 Errazuriz	AERRAZUR	12000	0.3	10-MAR-22	80
9	148 John	GCAMBRAU	11000	0.3	15-OCT-23	80
10	149 Peter	ELZLOTKE	10500	0.2	29-JAN-24	80

Deskripsi :

Perintah ini mengeksekusi pembuatan tabel employees secara dinamis dengan menyalin struktur serta seluruh baris data dari tabel hremployees. Secara teknis, ini adalah metode kloning cepat (*shallow copy*) yang memungkinkan kita melakukan manipulasi data tanpa risiko merusak tabel master. Hasil verifikasi menunjukkan data tersalin secara utuh sesuai dengan status terakhir pada tabel sumber.

2. Arsitektur Prosedur (increase_salary)

```
CREATE OR REPLACE PROCEDURE increase_salary (  
    department_id_in IN employees.department_id%TYPE,  
    increase_pct_in IN NUMBER  
) IS  
BEGIN  
    FOR employee_rec IN (SELECT employee_id FROM employees WHERE department_id = department_id_in)  
    LOOP  
        UPDATE employees emp  
        SET emp.salary = emp.salary + (emp.salary * increase_pct_in)  
        WHERE emp.employee_id = employee_rec.employee_id;  
  
        DBMS_OUTPUT.PUT_LINE('Updated ' || SQL%ROWCOUNT);  
    END LOOP;  
END increase_salary;
```

Deskripsi:

Stored procedure ini dirancang untuk melakukan pembaruan data gaji secara massal dan modular. Keunggulan kodenya terletak pada penggunaan %TYPE yang menjamin sinkronisasi tipe data parameter dengan kolom tabel. Logika FOR LOOP di dalamnya memastikan bahwa kalkulasi kenaikan gaji dilakukan secara spesifik hanya pada entitas yang memiliki kecocokan ID departemen, sehingga menjaga akurasi data.

3. Simulasi Pembatalan (Rollback)



```
BEGIN
    increase_salary (90, 0.50);
    ROLLBACK;
END;
/
SELECT * FROM employees WHERE department_id = 90;
```

Query Result x

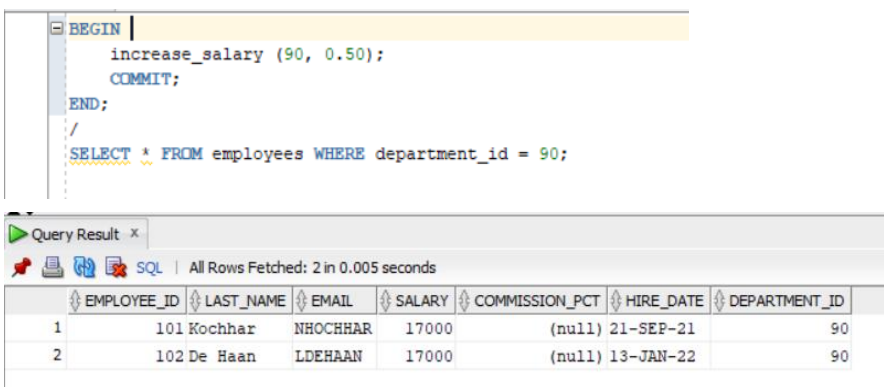
SQL | All Rows Fetched: 2 in 0.007 seconds

	EMPLOYEE_ID	LAST_NAME	EMAIL	SALARY	COMMISSION_PCT	HIRE_DATE	DEPARTMENT_ID
1	101	Kochhar	NHOCHHAR	17000	(null)	21-SEP-21	90
2	102	De Haan	LDEHAAN	17000	(null)	13-JAN-22	90

Deskripsi:

Langkah ini mendemonstrasikan aspek integritas transaksi. Meskipun prosedur `increase_salary` berhasil melakukan perhitungan untuk menaikkan gaji sebesar 50%, perintah `ROLLBACK` memerintahkan mesin database untuk mengabaikan perubahan tersebut. Hasilnya, data pada tabel tetap pada nilai semula (*baseline*) karena transaksi tidak difinalisasi ke dalam penyimpanan permanen.

4. Implementasi Final (Commit)



```
BEGIN
    increase_salary (90, 0.50);
    COMMIT;
END;
/
SELECT * FROM employees WHERE department_id = 90;
```

Query Result x

SQL | All Rows Fetched: 2 in 0.005 seconds

	EMPLOYEE_ID	LAST_NAME	EMAIL	SALARY	COMMISSION_PCT	HIRE_DATE	DEPARTMENT_ID
1	101	Kochhar	NHOCHHAR	17000	(null)	21-SEP-21	90
2	102	De Haan	LDEHAAN	17000	(null)	13-JAN-22	90

Deskripsi:

Pada tahap eksekusi final, perubahan diterapkan dan diakhiri dengan perintah `COMMIT`. Hal ini menyebabkan transformasi nilai pada kolom salary menjadi permanen secara fisik pada disk. Berdasarkan audit akhir, karyawan pada departemen 90 kini memiliki nilai gaji yang telah diperbarui sesuai dengan persentase yang diinputkan, menandakan sistem otomatisasi telah bekerja dengan sukses.